

TRINITY COLLEGE · SEPTEMBER 14, 2026

# AI Integration

## Your first coding agent

Ken Kousen · CPSC 415  
Monday 1:30–4:10 · MC-225

LIVE DEMONSTRATION

# An idea becomes something you can use

Help me memorize a text.

One line at a time.

A Next button.

Start over after the last line.

Watch the files change. Then try the result.

# What you bring to the work

## The agent

- Reads files
- Writes code
- Runs available tools
- Proposes changes

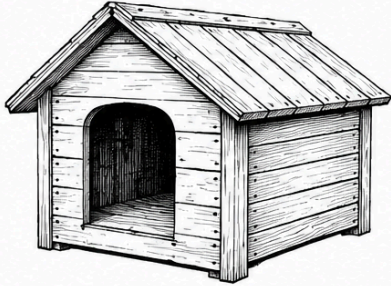
## You

- Define the outcome
- Choose and refine
- Check what happened
- Explain the result



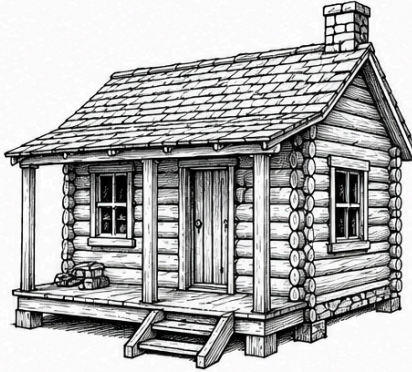
A working page is the beginning of the conversation.

# A small start, increasing responsibility



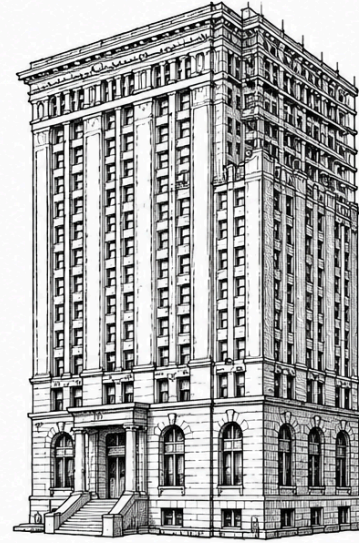
**Doghouse**

Make an idea tangible.



**Cabin**

Make it dependable.



**Skyscraper**

Coordinate larger systems.

# Model, harness, application

| Part               | Today                                          |
|--------------------|------------------------------------------------|
| <b>Model</b>       | The model selected through OpenRouter          |
| <b>Harness</b>     | Claude Code: context, tools, permissions, loop |
| <b>Application</b> | Your HTML page running in a browser            |

The agent uses AI to build the app.

The app itself makes no AI calls today.

## The agent works in a loop

Read → propose → act → observe

Then decide what to do next.



You can interrupt, correct the goal,  
or ask it to explain.

# Text becomes tokens

Welcome to Trinity Collge. Let's learn something about AI

| TOKENS | CHARACTERS |
|--------|------------|
| 12     | 57         |

Welcome to Trinity Collge. Let's learn something about AI

Words, word fragments, and punctuation.  
Different tokenizers can split the same text differently.

[The Tokenizer Playground](#)

# Know where the bill comes from

## OpenRouter

Your account  
Your selected model  
Your key's spending cap  
Your usage record

## Today's first candidate

`minimax/minimax-m3`

Pay-as-you-go API usage.  
Use the model announced in class.

If access fails, ask. Do not buy a subscription to fix it.



# Cost per accepted result

Calls + retries + corrections  
+ your time



Measure the work that reaches an acceptable result.

A free endpoint still costs time when it stalls.

A higher price still needs to earn its place.

2:00–2:45 · GUIDED SETUP

## Open the Week 1 guide

[github.com/kousen/ai-integration-course](https://github.com/kousen/ai-integration-course)

[weeks / 01](#)

Choose **macOS Terminal** or **Windows PowerShell**.  
Run one step at a time. Show the error if a step fails.

# Three setup checkpoints

| Checkpoint               | What you should see                          |
|--------------------------|----------------------------------------------|
| <b>A · Installed</b>     | Git and Claude Code print a version          |
| <b>B · Project ready</b> | Your own folder, poem, and Git branch        |
| <b>C · Connected</b>     | Agent reads 14 lines; correct model in usage |

Tell me the checkpoint where you are stuck.

# Keep two folders separate

```
cpsc415/  
  ai-integration-course/  ← guides and launcher  
  lyrics-trainer/        ← your files and commits  
    lyrics.txt  
    .gitignore
```

Start the agent inside `lyrics-trainer`.

# Launch through your wrapper

## macOS Terminal

```
bash ../ai-integration-course/scripts/orclaude "minimax/minimax-m3"
```

## Windows PowerShell

```
& ..\ai-integration-course\scripts\orclaude.bat "minimax/minimax-m3"
```

First enter your key using the hidden prompt in the guide.  
The instructor may substitute the rehearsed fallback model.

# Verify the connection

Read `lyrics.txt`.

How many nonblank lines does it contain?

Do not change any files yet.

1. Check the answer against the file: **14 lines**.
2. Inspect `/status`.
3. Check the model in OpenRouter Activity.

The agent's self-description is not billing evidence.

# Use only what this task needs

- Work inside your exercise folder.
- Use the supplied poem, not someone else's personal data.
- Keep the API key out of files, screenshots, and prompts.
- Read a proposed action before approving it.



A model can suggest something you should decline.

# Build the first version

Read the supplied poem.

Show one line and a counter.

Next advances; the last line wraps to the first.

One HTML file. No packages. No API calls.

Show a short plan and wait for approval.

The complete prompt is in [the lab handout](#).



# Choose one improvement

Previous · Restart · Keyboard shortcuts  
Readability



Decide the behavior before asking for the change.

Previous on line 1: stay there, or wrap to line 14?

# A useful change request

Add a Restart button.

It returns to line 1 and updates the counter.

Keep Next and wraparound working.

Keep the app in one file.

Explain the change before editing.

What stays the same is part of the requirement.

# Check the boundaries

| Action                  | Expected result              |
|-------------------------|------------------------------|
| Open the page           | First line; counter 1 of 14  |
| Next once               | Second line; counter 2 of 14 |
| Next from the last line | First line; counter 1 of 14  |
| Keyboard activation     | The focused button works     |
| Your change at an edge  | The behavior you specified   |

Record the actual result in `CHECKS.md`.

# When a check fails

## Report

What you did  
What you expected  
What you observed

## Verify the correction

Read the explanation  
Try the failing case again  
Check related behavior



If nothing fails, add a check. Do not invent a failure.

## **Explain one piece of the code**

Where is the current line stored?

What happens after the last line?

How does the counter change?

Point to the implementation, then show its behavior.

# Save evidence of the work

|                         |                                     |
|-------------------------|-------------------------------------|
| <code>index.html</code> | The app with your improvement       |
| <code>lyrics.txt</code> | The supplied text                   |
| <code>CHECKS.md</code>  | Expected and observed behavior      |
| <code>README.md</code>  | How to run it; choices and learning |
| <code>.gitignore</code> | Keep local credentials out          |

Commit the work. Tag it `week01-submitted`.

# The rest of the semester

| Component                               | Weight    |
|-----------------------------------------|-----------|
| Participation and labs                  | 15%       |
| Academy certificates                    | 10%       |
| Team projects: multimodal / agentic-MCP | 15% / 20% |
| Individual AI-enabled profile site      | 25%       |
| Individual Comprehension Defense        | 15%       |

Choose and justify your project languages.  
One guided exercise will explore an unfamiliar language.

# The process grows with the work

| Week     | New evidence                              |
|----------|-------------------------------------------|
| 1        | Working app, checks, explanation, commits |
| 2–3      | Intent, then specification                |
| 4–5      | Written plan, then automated checks       |
| 6 onward | Reviewed pull requests and the full loop  |

Write down decisions while they are being made.



# Before next Monday

**September 21 · 1:30 PM**

- Submit your repository URL and `week01-submitted` tag.
- Upload your **Claude Code 101** certificate.
- Read the syllabus and submit the **Academic Honesty Pledge**.

**Start now:** AI Fluency for Students, due September 28.

Week 1 guides and public examples are linked from Moodle.  
Selected draft book readings will be shared there separately.

## Before you leave

What did you ask for?

What did you change?

What did you check?



Show the result, and one thing you understand about it.

OPTIONAL INSTRUCTOR DEMONSTRATION · TIME PERMITTING

## Three interfaces, one application

Compare. Choose. Refine.

Then check the behavior again.

Ken demonstrates Anthropic's `/design` using his own access.  
This command is not part of the OpenRouter lab.

# Sources and next steps

- Ken Kousen, *Claude Code: Up and Running*: Chapters 1–4 and selected Chapter 11 material. Draft readings via Moodle.
- [Companion examples and launcher](#)
- [Claude Code training](#) · [Codex training](#)
- [Practical AI Literacy](#): tokenizer screenshots and cost/verification teaching material
- [Tokenizer Playground](#)
- [Claude Code setup](#) · [OpenRouter integration](#)
- Shakespeare, Sonnet 18: public-domain lab text. Project-scale illustration from Ken's book, used for this course.

Prepared September 9, 2026. Recheck models and service interfaces before class.